



COM410 Programming in Practice

T6. Developing Secure Code



Secure Implementation

- Programmers must include **fail-safe** measures in their code to make programs secure for their users
- Java manages memory for you, checks validity of array indices, and is type-safe
- However, a mistake can make code vulnerable
- You can practice fail-safe programming by including checks for anticipated errors
- When implementing an ADT, ask the following questions:
 1. What happens if a constructor does not execute completely?
 2. What might happen if a client tries to exceed the storage limit when creating a collection type?

Securing the ArrayBag

- For the `ArrayBag` class we want to guard against both these situations
- We begin by refining the implementation to make code more secure by adding two additional data fields to the class:

```
...  
private T[] bag;  
private int numberOfEntries;  
private static final int DEFAULT_CAPACITY = 25;  
private boolean initialised = false;  
private static final int MAX_CAPACITY = 10000;  
...
```

→ New instance variable

→ New class variable

- To ensure a client cannot create a bag that is too large, the constructor should check the desired capacity (passed as a parameter) against **MAX_CAPACITY**
- If the request is too large, it can throw an exception

Securing the Implementation

- If the requested capacity is within range, the constructor could still fail (array allocation could fail due to insufficient memory)
- To prevent this, each vital method of the class can check the status field
- The constructor should set **initialised** to **true** for correctly initialised objects:

```
public ArrayBag(int capacity) {  
    if (capacity <= MAX_CAPACITY) {  
        T[] tempBag = (T[]) new Object[capacity];  
        bag = tempBag;  
        numberOfEntries = 0;  
        initialised = true;  
    } else throw new IllegalStateException(  
        "Attempt to create a bag where the capacity  
        exceeds the maximum");  
}
```

Securing the Implementation

- Since `initialised` will be checked in several methods, repetitive code can be avoided by defining a private method:

```
private void checkInitialisation() {  
    if (!initialised)  
        throw new SecurityException(  
            "ArrayBag object is not initialised properly");  
}
```

- The method can be called by other methods in the class prior to performing their operations

```
public boolean addNewEntry(T newEntry) {  
    checkInitialisation();  
    ...  
}
```

- Note: the exceptions `SecurityException` and `IllegalStateException` are standard runtime exceptions (in package `java.lang`.)

Securing the Implementation

- Some common guidelines for writing more secure Java code:
 1. Declare most (if not all) data fields of a class as private (any public data fields should be static and final and have constant values)
 2. Don't be too clever!
 3. Avoid duplicate code; instead, encapsulate such code into a private method that other methods can call
 4. When a constructor calls a method, ensure it cannot be overridden
- As such, we revise the declaration for class **ArrayBag**:

```
public final class ArrayBag<T> implements BagInterface<T> {
```

- As it is declared as **final** no other class can extend it
- A **final class** is more secure because it cannot be inherited to change its behaviour

Scenario

- Secure your implementation of the `ArrayBag` class by adding the following
 - i. Add the `initialised` and `MAX_CAPACITY` variables
 - ii. Update the constructor to check that the requested capacity is allowed and throw an exception if not. If the Bag is created, the `initialised` class variable should be set to `true`
 - iii. Add the new private method `checkInitialisation()` to test the value of `initialised` and throw an exception if it is `false`
 - iv. Call the new `checkInitialisation()` method as the first action in the `addNewEntry()` method
 - v. Update the class header to define the `ArrayBag` class as `final`
- Check that your implementation still works as before following these changes